SQL JOIN Questions & Solutions

Farmer's Market Dataset

Introduction

This document provides a set of descriptive SQL JOIN questions and their solutions, based on the farmer's market dataset.

---

1 INNER JOIN Questions

1. **Customers who made a purchase: Find the first and last name of all customers who appear in the `customer_purchases` table. The list should not contain duplicates.**

```sql
SELECT DISTINCT
    c.customer_first_name,
    c.customer_last_name
FROM customer c
JOIN customer_purchases cp ON c.customer_id = cp.customer_id;
```

2. **Product name and category for products sold: List the unique `product_name` and `product_category_name` for every product that has at least one transaction in the `customer_purchases` table.**

```sql
SELECT DISTINCT
    p.product_name,
    pc.product_category_name
FROM customer_purchases cp
JOIN product p ON cp.product_id = p.product_id
JOIN product_category pc ON p.product_category_id = pc.product_category_id;
```

3. **Market date, vendor, and booth description: Show the `market_date`, `vendor_name`, and `booth_description` for all booth assignments recorded in the `vendor_booth_assignments` table.**

```sql
SELECT
    vba.market_date,
    v.vendor_name,
    b.booth_description
FROM vendor_booth_assignments vba
JOIN vendor v ON vba.vendor_id = v.vendor_id
JOIN booth b ON vba.booth_number = b.booth_number;
```

4. **Products sold by "Marco's Peppers": Find all unique `product_names` that were sold by the vendor named 'Marco's Peppers'.**

```sql
SELECT DISTINCT
    p.product_name
FROM customer_purchases cp
JOIN product p ON cp.product_id = p.product_id
JOIN vendor v ON cp.vendor_id = v.vendor_id
WHERE v.vendor_name = 'Marco''s Peppers';
-- Note: The single quote in 'Marco's' must be escaped with another single quote.
```

5. **Total revenue by vendor type: Calculate the total revenue (defined as `quantity` × `cost_to_customer_per_qty`) for each `vendor_type`.**

```sql
SELECT
    v.vendor_type,
    SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_revenue
FROM customer_purchases cp
JOIN vendor v ON cp.vendor_id = v.vendor_id
GROUP BY v.vendor_type;
```

6. **Customer, product for purchases on '2019-07-06': List the `customer_first_name`, `customer_las` and `product_name` for all transactions that occurred on the date '2019-07-06'.**

```sql
SELECT
    c.customer_first_name,
    c.customer_last_name,
    p.product_name
FROM customer_purchases cp
JOIN customer c ON cp.customer_id = c.customer_id
JOIN product p ON cp.product_id = p.product_id
WHERE cp.market_date = '2019-07-06';
```

7. **Fresh Fruits & Veg in inventory in August 2019: Find all unique `product_names` that are in the 'Fresh Fruits & Vegetables' category AND were present in `vendor_inventory` at any point in August 2019.**

```sql
SELECT DISTINCT
    p.product_name
FROM product p
JOIN product_category pc ON p.product_category_id = pc.product_category_id
JOIN vendor_inventory vi ON p.product_id = vi.product_id
WHERE
    pc.product_category_name = 'Fresh Fruits & Vegetables'
    AND vi.market_date LIKE '2019-08%'; -- Assumes date is stored as text
```

8. **Customers who bought 'lbs' products: List the unique names (`customer_first_name`, `customer_last_name`) of customers who have purchased at least one product where the `product_qty_type` is 'lbs'.**

```sql
SELECT DISTINCT
    c.customer_first_name,
    c.customer_last_name
FROM customer c
JOIN customer_purchases cp ON c.customer_id = cp.customer_id
JOIN product p ON cp.product_id = p.product_id
WHERE p.product_qty_type = 'lbs';
```

9. **4-table join (customer, vendor, product, booth): Write a query that links a customer's purchase to the booth they bought it from. Show the `customer_first_name`, `vendor_name`, `product_name`, and `booth_price_level` for all transactions. (Hint: A transaction is linked to a booth via the vendor AND the market date).**

```sql
-- This query links transactions to booth assignments via vendor and date.
SELECT
    c.customer_first_name,
    v.vendor_name,
    p.product_name,
    b.booth_price_level
FROM customer_purchases cp
JOIN customer c ON cp.customer_id = c.customer_id
JOIN vendor v ON cp.vendor_id = v.vendor_id
JOIN product p ON cp.product_id = p.product_id
```

```
JOIN vendor_booth_assignments vba ON cp.vendor_id = vba.vendor_id
                                 AND cp.market_date = vba.market_date
JOIN booth b ON vba.booth_number = b.booth_number;
```

---

2 LEFT & RIGHT JOIN Questions

1. **Customers who have never made a purchase: Find the names (`customer_first_name`, `customer_last_name`) of all customers in the `customer` table who have no matching records in the `customer_purchases` table.**

```
SELECT
    c.customer_first_name,
    c.customer_last_name
FROM customer c
LEFT JOIN customer_purchases cp ON c.customer_id = cp.customer_id
WHERE cp.customer_id IS NULL;
```

2. **All vendors and their assigned booth count: List all vendors from the `vendor` table. For each vendor, show their name and a count of how many booth assignments they have in `vendor_booth_assignments`. Vendors who were never assigned a booth should be listed with a count of 0.**

```
SELECT
    v.vendor_name,
    COUNT(vba.booth_number) AS booth_assignment_count
FROM vendor v
LEFT JOIN vendor_booth_assignments vba ON v.vendor_id = vba.vendor_id
GROUP BY v.vendor_id, v.vendor_name;
-- COUNT(column_name) ignores NULLs, so vendors with 0 assignments get 0.
```

3. **Vendors not assigned a "Standard" booth: Find the names of all vendors who have never been assigned to a booth with the `booth_type` 'Standard'. This includes vendors who have no assignments at all.**

```
SELECT v.vendor_name
FROM vendor v
LEFT JOIN (
    -- Subquery to find all vendors who HAVE been in a 'Standard' booth
    SELECT DISTINCT vba.vendor_id
    FROM vendor_booth_assignments vba
    JOIN booth b ON vba.booth_number = b.booth_number
    WHERE b.booth_type = 'Standard'
) AS standard_vendors ON v.vendor_id = standard_vendors.vendor_id
WHERE standard_vendors.vendor_id IS NULL;
```

4. **All market dates and vendor count: List all market dates from `market_date_info` and show the number of vendors who were assigned a booth on that day. Dates with no vendors should still be listed, showing a count of 0.**

```
SELECT
    mdi.market_date,
    COUNT(vba.vendor_id) AS number_of_vendors
FROM market_date_info mdi
LEFT JOIN vendor_booth_assignments vba ON mdi.market_date = vba.market_date
GROUP BY mdi.market_date;
```

5. **Products and their first-ever purchase date: List all `product_names`. For each product, show the date of its first-ever sale (i.e., the $MIN(market_date)$ from `customer_purchases`). Products that were never sold should still be in the list, with their first purchase date shown as $NULL$.**

```sql
SELECT
    p.product_name,
    MIN(cp.market_date) AS first_purchase_date
FROM product p
LEFT JOIN customer_purchases cp ON p.product_id = cp.product_id
GROUP BY p.product_id, p.product_name;
```

6. **Customers not purchasing 'Eggs & Meat...': Find all customers who have never purchased a product from the 'Eggs & Meat (Fresh or Frozen)' category. This must be done without using $NOTEXISTS$ (i.e., use $NOTIN$ with a subquery).**

```sql
SELECT
    c.customer_first_name,
    c.customer_last_name
FROM customer c
WHERE c.customer_id NOT IN (
    -- Subquery to get all customers who HAVE bought from this category
    SELECT DISTINCT cp.customer_id
    FROM customer_purchases cp
    JOIN product p ON cp.product_id = p.product_id
    JOIN product_category pc ON p.product_category_id = pc.product_category_id
    WHERE pc.product_category_name = 'Eggs & Meat (Fresh or Frozen)'
);
-- An alternative LEFT JOIN / IS NULL is also a valid solution.
```

7. **Inventory on '2019-08-03' but zero sales on that date: Find all `product_ids` and `vendor_ids` that were in `vendor_inventory` on '2019-08-03' but had zero sales (no matching record in `customer_purchases`) on that exact same date.**

```sql
SELECT
    vi.product_id,
    vi.vendor_id
FROM vendor_inventory vi
LEFT JOIN customer_purchases cp ON vi.vendor_id = cp.vendor_id
                               AND vi.product_id = cp.product_id
                               AND vi.market_date = cp.market_date
WHERE
    vi.market_date = '2019-08-03'
    AND cp.customer_id IS NULL; -- The join condition failed = no purchase
```

---

3 SELF JOIN Questions

1. **Pairs of customers in the same $customer_zip$: Find all unique pairs of customers (e.g., (Jane, Bob) but not (Bob, Jane)) who live in the same `customer_zip`. Do not list a customer paired with themselves.**

```sql
SELECT
    c1.customer_first_name, c1.customer_last_name,
    c2.customer_first_name, c2.customer_last_name,
    c1.customer_zip
FROM customer c1
JOIN customer c2 ON c1.customer_zip = c2.customer_zip
                AND c1.customer_id < c2.customer_id;
-- c1.customer_id < c2.customer_id prevents self-pairs and duplicate pairs.
```

2. **Products in the same category as "Apple Pie": Use a $SELFJOIN$ on the `product` table to find all `product_names` that share the same `product_category_id` as 'Apple Pie'. Exclude 'Apple Pie' itself from the results.**

```sql
SELECT
    p1.product_name
FROM product p1
JOIN product p2 ON p1.product_category_id = p2.product_category_id
WHERE
    p2.product_name = 'Apple Pie'
    AND p1.product_name <> 'Apple Pie';
```

3. **Market dates in the same week/year as '2019-03-16': Use a $SELFJOIN$ on `market_date_info` to find all `market_dates` that occurred in the same `market_week` and `market_year` as '2019-03-16'. Exclude '2019-03-16' from the list.**

```sql
SELECT
    m1.market_date
FROM market_date_info m1
JOIN market_date_info m2 ON m1.market_week = m2.market_week
                         AND m1.market_year = m2.market_year
WHERE
    m2.market_date = '2019-03-16'
    AND m1.market_date <> '2019-03-16';
```

4. **Pairs of vendors of the same $vendor_type$: Find all unique pairs of `vendor_names` (e.g., (A, B) but not (B, A) or (A, A)) who share the same `vendor_type`.**

```sql
SELECT
    v1.vendor_name,
    v2.vendor_name,
    v1.vendor_type
FROM vendor v1
JOIN vendor v2 ON v1.vendor_type = v2.vendor_type
               AND v1.vendor_id < v2.vendor_id;
```

5. **Pairs of customers who bought the same item: Find all unique pairs of `customer_ids` who bought the exact same `product_id`, from the same `vendor_id`, on the same `market_date`.**

```sql
SELECT DISTINCT
    c1.customer_id,
    c2.customer_id
FROM customer_purchases c1
JOIN customer_purchases c2 ON c1.product_id = c2.product_id
                          AND c1.market_date = c2.market_date
                          AND c1.vendor_id = c2.vendor_id
                          AND c1.customer_id < c2.customer_id;
```

---

4 Handling $NULL$s, Blanks, and Duplicates

1. **$NULL$ and 'Null' string in $product_qty_type$: The `product` table has both $NULL$ values and the literal string 'Null' in the `product_qty_type` column. Write a query to find all transaction details from `customer_purchases` for products where this value is either $NULL$ or the string 'Null'.**

```sql
SELECT
    p.product_name,
    p.product_qty_type,
    cp.customer_id,
    cp.quantity
FROM product p
JOIN customer_purchases cp ON p.product_id = p.product_id
WHERE
    p.product_qty_type IS NULL
    OR p.product_qty_type = 'Null';
```

2. $NULL$, **blank, 'null' in** $product_size$**: The `product_size` column has** $NULL$ **values, blank strings (' '), and the literal string 'null'. Write a query to count the number of products per `product_category_name` that have one of these "unknown" sizes.**

```sql
SELECT
    pc.product_category_name,
    COUNT(p.product_id) AS product_count
FROM product_category pc
JOIN product p ON pc.product_category_id = p.product_category_id
WHERE
    p.product_size IS NULL
    OR p.product_size = ' '       -- Blank string
    OR p.product_size = ''        -- Empty string (some SQL dialects)
    OR p.product_size = 'null'  -- The string 'null'
GROUP BY pc.product_category_name;
```

3. **Duplicates in** $vendor_booth_assignments$**: The `vendor_booth_assignments` table can assign the same vendor to multiple booths on the same date (e.g., vendor 4 on 2020-08-26). Write a query to show the `vendor_name` only once, along with a count of their booths for that day.**

```sql
SELECT
    v.vendor_name,
    vba.market_date,
    COUNT(vba.booth_number) AS booth_count
FROM vendor v
JOIN vendor_booth_assignments vba ON v.vendor_id = vba.vendor_id
WHERE
    v.vendor_id = 4
    AND vba.market_date = '2020-08-26'
GROUP BY v.vendor_id, v.vendor_name, vba.market_date;
```

---

5 Complex & Other JOINs

1. $CROSS JOIN$**: Write a query to produce every possible pairing of a `vendor_name` (vendor) and a `booth_description` (booth).**

```sql
SELECT
    v.vendor_name,
    b.booth_description
FROM vendor v
CROSS JOIN booth b;
```

2. $ANTI JOIN$ **(NOT EXISTS): Write a query using** $NOT EXISTS$ **to emulate an** $ANTI JOIN$ **that finds all customers (`customer`) who have never purchased 'Sweet Corn' (`product`, `customer_purchases`).**

```sql
SELECT c.customer_first_name, c.customer_last_name
FROM customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM customer_purchases cp
    JOIN product p ON cp.product_id = p.product_id
    WHERE cp.customer_id = c.customer_id
    AND p.product_name = 'Sweet Corn'
);
```

3. **Top selling vendor on '2019-07-06': Find the `vendor_name` who sold the most (by total revenue: `quantity` $\times$ `cost_to_customer_per_qty`) on '2019-07-06'.**

```sql
SELECT
    v.vendor_name,
    SUM(cp.quantity * cp.cost_to_customer_per_qty) AS total_sales
FROM vendor v
JOIN customer_purchases cp ON v.vendor_id = cp.vendor_id
WHERE cp.market_date = '2019-07-06'
GROUP BY v.vendor_id, v.vendor_name
ORDER BY total_sales DESC
LIMIT 1; -- (Use TOP 1 in SQL Server)
```

4. ***product_id*s in *vendor_inventory* but never sold: Find all unique `product_id`s that are present in the `vendor_inventory` table but are not present in the `customer_purchases` table at all.**

```sql
SELECT DISTINCT product_id
FROM vendor_inventory
WHERE product_id NOT IN (
    SELECT DISTINCT product_id
    FROM customer_purchases
);
```

5. **Vendors assigned to every *booth_type*: (Relational Division) Find the `vendor_name` of any vendor who has been assigned to all of the distinct *booth_type*s available in the `booth` table (i.e., 'Standard', 'Small', AND 'Large').**

```sql
SELECT
    v.vendor_name
FROM vendor v
JOIN vendor_booth_assignments vba ON v.vendor_id = vba.vendor_id
JOIN booth b ON vba.booth_number = b.booth_number
GROUP BY v.vendor_id, v.vendor_name
HAVING
    COUNT(DISTINCT b.booth_type) = (SELECT COUNT(DISTINCT booth_type) FROM booth);
```

6. **Inventory item sold (by anyone) on same day: List the `market_date`, `vendor_name`, and `product_name` for every item in a vendor's inventory, but only if that same product was sold by any vendor on that same day.**

```sql
SELECT DISTINCT
    vi.market_date,
    v.vendor_name,
    p.product_name
FROM vendor_inventory vi
JOIN vendor v ON vi.vendor_id = v.vendor_id
JOIN product p ON vi.product_id = p.product_id
WHERE EXISTS (
    -- Check if ANYONE sold this product on this day
    SELECT 1
    FROM customer_purchases cp
```

```
    WHERE cp.product_id = vi.product_id
    AND cp.market_date = vi.market_date
);
```

7. **Vendor loyalty (booth count in 2019): List all vendors and count how many distinct $market_date$s they were assigned a booth during the 2019 $market_year$. Include vendors who had 0 assignments in 2019.**

```
SELECT
    v.vendor_name,
    COUNT(DISTINCT vba_2019.market_date) AS market_day_count
FROM vendor v
LEFT JOIN (
    -- Pre-filter assignments for 2019
    SELECT vba.vendor_id, vba.market_date
    FROM vendor_booth_assignments vba
    JOIN market_date_info mdi ON vba.market_date = mdi.market_date
    WHERE mdi.market_year = 2019
) AS vba_2019 ON v.vendor_id = vba_2019.vendor_id
GROUP BY v.vendor_id, v.vendor_name
ORDER BY market_day_count DESC;
```

8. $NATURALJOIN$: **Using a `NATURAL JOIN`, combine the `product` and `product_category` tables to show the `product_name` and `product_category_name`. (Note: This join works because the only column name they share is `product_category_id`).**

```
SELECT
    product_name,
    product_category_name
FROM product
NATURAL JOIN product_category;
```

9. $FULLOUTERJOIN$: **List all `product_ids` from `vendor_inventory` and `customer_purchases` for the date '2019-07-03'. Show the `product_id` from the inventory table and the `product_id` from the purchases table to demonstrate how non-matches are shown.**

```
SELECT
    inv.product_id AS inventory_product,
    purch.product_id AS purchase_product
FROM (
    SELECT DISTINCT product_id FROM vendor_inventory WHERE market_date = '2019-07-03
    '
) AS inv
FULL OUTER JOIN (
    SELECT DISTINCT product_id FROM customer_purchases WHERE market_date = '
    2019-07-03'
) AS purch ON inv.product_id = purch.product_id;
-- Rows with NULL on one side were in one table, but not the other.
```